

DevOps Journey - Day 1: Shell Scripting

Suraj Rane

Day 1: Shell Scripting - Basics Revision, Intermediate Concepts, Exit Status & Assignment

Part 1: Quick Basics Revision

1.1 Shebang & Running a Script

```
#!/bin/bash

chmod +x script.sh
./script.sh
```

1.2 Variables

```
name="Suraj"
echo "$name"      # always quote variables
echo "Hello, $name"
```

No spaces around =. Use \$name or \${name} to reference.

1.3 Input

```
echo "Enter name:"
read name

# script arguments
echo "$1"      # first arg
echo "$@"      # all args
echo "$#"      # number of args
```

1.4 Conditionals

```
if [ "$age" -gt 18 ]; then
    echo "Adult"
elif [ "$age" -eq 18 ]; then
    echo "Just 18"
else
    echo "Minor"
fi
```

Common test flags:

Numbers	Strings	Files
-eq -ne	= !=	-f (file exists)
-gt -lt	-z (empty)	-d (dir exists)
-ge -le	-n (not empty)	-x (executable)

1.5 Loops

```
for i in 1 2 3; do
    echo "$i"
done

for file in *.txt; do
    echo "$file"
done

i=0
while [ $i -lt 5 ]; do
    echo "$i"
    i=$((i+1))
done
```

1.6 Functions

```
greet() {
    echo "Hello, $1"
}
greet "Suraj"
```

1.7 Arithmetic

```
a=5
b=3
echo $((a + b))
echo $((a * b))
```

1.8 Common Gotchas

- Always quote variables: "\$var" not \$var (breaks on spaces otherwise)
 - [] needs spaces inside: ["\$x" -eq 1] not ["\$x" -eq 1]
 - \$(command) for command substitution, not backticks
 - Comparing strings: =, comparing numbers: -eq
-

Part 2: Intermediate Concepts

2.1 Safe Script Header

Every script should start with:

```
#!/bin/bash
set -euo pipefail
```

- set -e - exit immediately if any command fails (instead of silently continuing)

- `set -u` - treat unset variables as an error (catches typos like `$FIL` instead of `$FILE`)
- `set -o pipefail` - if any command in a pipeline fails, the whole pipeline is considered failed (without this, `cmd1 | cmd2` only checks `cmd2`'s exit code)

This one line prevents a huge class of “script silently did the wrong thing” bugs - very relevant to production scripts.

2.2 Arrays

```
servers=("web01" "web02" "db01")
echo "${servers[0]}"           # web01
echo "${servers[@]}"           # all elements
echo "${#servers[@]}"          # count -> 3

for s in "${servers[@]}; do
    echo "Checking $s"
done
```

2.3 String Manipulation (built-in, no external tools needed)

```
file="server_logs_2026.txt"
echo "${file%.txt}"            # remove suffix -> server_logs_2026
echo "${file#server_}"         # remove prefix -> logs_2026.txt
echo "${file/2026/2025}"       # replace -> server_logs_2025.txt
echo "${#file}"                # length
```

2.4 grep, sed, awk Combined

Say you have a log file and want all ERROR lines, extract just the message, and count occurrences per hour.

```
# Extract ERROR lines
grep "ERROR" app.log

# grep + sed: pull out just the message after the ERROR tag
grep "ERROR" app.log | sed 's/.*ERROR: //'

# awk: print specific columns (log format: timestamp level message)
awk '$2=="ERROR" {print $1, $3}' app.log

# Count errors per hour (timestamp like 14:32:01)
grep "ERROR" app.log | awk -F: '{print $1}' | sort | uniq -c
```

2.5 Exit Codes - Basics

```
grep "FATAL" app.log
if [ $? -eq 0 ]; then
    echo "Fatal error found!"
else
    echo "No fatal errors"
fi

# cleaner way -- don't even need $?
if grep -q "FATAL" app.log; then
    echo "Fatal error found!"
```

```
fi
```

Custom exit codes in your own scripts:

```
if [ ! -f "$1" ]; then
    echo "File not found" >&2
    exit 1
fi
```

Part 3: Exit Status - Deep Dive

3.1 \$? - Exit Status of the Last Command

```
grep "ERROR" app.log
echo $?
```

- 0 = success
- Non-zero (1-255) = failure (different tools use different non-zero codes to mean different things)

Important: \$? only holds the exit code of the *most recently run* command - check it immediately after, since running another command overwrites it.

```
ls /nonexistent
echo $?      # 2 -> no such file/dir

ls /home
echo $?      # 0 -> success
```

3.2 Using It Directly in a Condition (cleaner, no \$? needed)

```
if grep -q "ERROR" app.log; then
    echo "Found errors"
else
    echo "No errors"
fi
```

This works because if just checks the exit status of the command directly.

3.3 Exit Status of a Specific Command in a Pipeline

```
cmd1 | cmd2 | cmd3
echo "${PIPESTATUS[@]}" # array of exit codes for each command in
the pipe
```

`${PIPESTATUS[0]}` = exit code of `cmd1`, `${PIPESTATUS[1]}` = exit code of `cmd2`, etc. \$? alone only tells you about the *last* command in the pipe - which is exactly why `set -o pipefail` exists, to make the whole pipeline fail if any part fails.

3.4 Setting Your Own Exit Status in a Script

```
#!/bin/bash
if [ ! -f "$1" ]; then
```

```
        echo "File not found" >&2
        exit 1
    fi
    exit 0
```

Whatever you pass to exit becomes that script's exit status, checkable via \$? by whoever calls it.

3.5 Checking Exit Status of a Background Job

```
some_long_command &
wait $!
echo $?
```

Summary: \$? for "just ran," PIPESTATUS for pipelines, exit N for setting your own.

Part 4: Hands-On Assignment

Setup - create a test log file

```
cat << 'EOF' > app.log
10:01:23 INFO Application started
10:02:15 ERROR Failed to connect to DB
10:05:44 WARN High memory usage
10:07:12 ERROR Timeout on request
11:15:33 ERROR Failed to connect to DB
11:20:01 INFO Request processed
EOF
```

Task: Write error_summary.sh

Your script must:

1. Use `set -euo pipefail`
2. Take a log filename as \$1, exit with code 1 and an error message if the file doesn't exist
3. Print total count of ERROR lines
4. Print count of ERROR lines grouped by hour
5. Print just the unique error messages (deduplicated)

Tips for approaching it: - Combine techniques from Part 2 (arrays, grep/sed/awk) with the exit-status handling from Part 3 - Test both the success path (file exists) and the failure path (file missing) - check \$? after running it both ways - Keep quoting disciplined (" \$1 " not \$1) per the gotchas in Part 1

End of Day 1 - DevOps Journey (55-Day Roadmap)